

UD2

PUNTOS COMPLETOS DE PHP

Programación en servidor

Desde variables hasta validación de formularios

MP_0613

Desarrollo web
en entorno
servidor

Contenido

Antes de empezar: cómo probar el código	3
1. Sintaxis básica	3
2. Variables y tipos de datos	4
3. Operadores	6
4. Salida de datos	7
5. Estructuras de control	9
6. Bucles	10
7. Arrays	12
8. Funciones	14
9. Mezclando PHP y HTML	17
10. Formularios: \$_GET y \$_POST	18
11. Validación y sanitización de datos	20
12. Cadenas de texto	23
13. Resumen, errores típicos y buenas prácticas	25

Antes de empezar: cómo probar el código

Para practicar no hace falta instalar nada: el editor online gratuito de W3Schools ejecuta PHP en su servidor y devuelve el HTML resultante.

- Abre el navegador y entra en el **W3Schools Tryit Editor** (PHP).
- Borra el contenido de la izquierda, pega o escribe el código y pulsa el botón verde **Run**.
- El resultado aparece a la derecha. Es exactamente lo que haría un servidor propio.
- Alternativa en local: XAMPP/WAMP, guardar el archivo con extensión **.php** en la carpeta **htdocs** y abrir **http://localhost/archivo.php**.

Importante: PHP se ejecuta en el servidor. El navegador nunca recibe el código PHP, solo el HTML que genera. Por eso un archivo .php no se puede abrir con doble clic: tiene que pasar por un servidor.

1. Sintaxis básica

Un archivo PHP es un archivo HTML normal que puede contener bloques de código PHP. Cuando el servidor lo procesa, ejecuta ese código y lo sustituye por su resultado antes de enviarlo al navegador.

```
<?php
    echo "¡Hola, mundo!";      // comentario de una línea
    # también es un comentario de una línea
    /* comentario
       de varias líneas */
?>
```

- Todo el código PHP va entre **<?php** y **?>**.
- Cada instrucción termina con **punto y coma (;)**.
- Comentarios: **//** o **#** (una línea) y **/* ... */** (varias líneas).
- Si el archivo contiene solo PHP, se puede omitir la etiqueta de cierre **?>**.
- **<?= \$variable ?>** es una forma abreviada de **<?php echo \$variable; ?>**.
- Las llaves **{ }** agrupan bloques de instrucciones (if, bucles, funciones).

¿Distingue mayúsculas y minúsculas?

Elemento	¿Distingue?	Ejemplo
Nombres de variables	Sí	\$nombre y \$Nombre son variables distintas
Nombres de constantes	Sí	IVA y iva son distintas
Nombres de funciones	No	echo , ECHO y Echo son equivalentes
Palabras clave (if, else, while...)	No	IF equivale a if

2. Variables y tipos de datos

Las variables se declaran con el símbolo **\$** delante del nombre. No hace falta indicar el tipo: PHP lo deduce del valor asignado y puede cambiar durante la ejecución.

```
<?php
    $nombre  = "Ana";      // string
    $edad    = 21;         // int
    $altura  = 1.68;       // float
    $esAlumna = true;      // bool
    $sinValor = null;      // null

    echo $nombre;

?>
```

Reglas para los nombres de variables

- Empiezan por **\$** seguido de una letra o guion bajo (**\$_total**). **Nunca por un número.**
- Solo contienen letras, números y guiones bajos. Sin espacios ni guiones medios.
- Distinguen mayúsculas de minúsculas.
- Usa nombres descriptivos: **\$precioTotal** mejor que **\$x**.

Tipos de datos básicos

Tipo	Qué almacena	Ejemplo
string	Texto entre comillas simples o dobles	"Hola", 'PHP'
int	Números enteros	42, -7, 0
float	Números decimales (con punto)	3.14, -0.5
bool	Verdadero o falso	true, false
array	Colección de valores	[1, 2, 3]
null	Ausencia de valor	null

Comprobar el tipo de una variable

```
<?php
    $x = 5;
    echo gettype($x);      // integer
    var_dump($x);          // int(5)
    var_dump(3.14);        // float(3.14)
    var_dump("hola");      // string(4) "hola"
    var_dump(true);        // bool(true)

    var_dump(is_int($x));   // true
    // is_string(), is_float(), is_bool(), is_array(), is_null(), is_numeric()

?>
```

Conversión de tipos (casting)

```
<?php
    var_dump((int) "12abc");    // int(12)
    var_dump((int) 3.99);      // int(3)   (trunca, no redondea)
    var_dump((float) "3.5");   // float(3.5)
    var_dump((string) 45);     // string(2) "45"
    var_dump((bool) "0");      // bool(false)
    var_dump((bool) "hola");   // bool(true)
    var_dump(intval("abc"));    // int(0)
    var_dump("5" + 3);          // int(8)   PHP convierte "5" a número
    var_dump("5" . 3);          // string(2) "53" el punto concatena
?>
```

isset(), empty() e is_null()

Valor de \$v	isset(\$v)	empty(\$v)	is_null(\$v)
null	false	true	true
0	true	true	false
"0"	true	true	false
"" (cadena vacía)	true	true	false
[] (array vacío)	true	true	false
false	true	true	false
"hola", 5	true	false	false

- **isset(\$v)**: la variable existe y **no es null**.
- **empty(\$v)**: la variable no existe o vale 0, "0", "", null, false o [].
- **unset(\$v)** elimina la variable.

Constantes

Una constante guarda un valor que no puede cambiar. Se escribe **sin \$** y por convención en mayúsculas.

```
<?php
    define("IVA", 21);          // con define()
    const PAIS = "España";      // con const

    echo IVA;                   // 21
    echo PHP_EOL;               // constantes predefinidas: PHP_EOL, PHP_INT_MAX, PHP_VERSION, M_PI
    echo M_PI;                   // 3.1415926535898
?>
```

3. Operadores

Aritméticos

Operador	Nombre	Ejemplo	Resultado
+	Suma	7 + 2	9
-	Resta	7 - 2	5
*	Multiplicación	7 * 2	14
/	División	7 / 2	3.5
%	Resto (módulo)	7 % 2	1
**	Potencia	2 ** 3	8

Importante: Dividir o calcular el resto entre 0 lanza un error (DivisionByZeroError). Comprueba siempre que el divisor no sea 0. Para dividir y obtener un entero usa `intval(7, 2)` que da 3.

Asignación, incremento y decremento

```
<?php
$x = 10;
$x += 5;    // $x = $x + 5  -> 15
$x -= 3;    // 12
$x *= 2;    // 24
$x /= 4;    // 6
$x %= 4;    // 2
$x .= "kg"; // concatena -> "2kg"

$i = 5;
$i++;       // 6 (post-incremento)
++$i;      // 7 (pre-incremento)
$i--;       // 6
?>
```

Comparación

Operador	Significado	Ejemplo	Resultado
==	Igual (solo valor)	5 == "5"	true
===	Idéntico (valor y tipo)	5 === "5"	false
!= o <>	Distinto (valor)	5 != "5"	false
!==	No idéntico	5 !== "5"	true
< > <= >=	Menor, mayor...	4 >= 4	true
<=>	Nave espacial (-1, 0, 1)	3 <=> 5	-1

```
<?php
$a = 5;
$b = "5";
var_dump($a == $b);    // true (mismo valor)
var_dump($a === $b);  // false (distinto tipo)

var_dump(0 == "");    // true en PHP 7, false en PHP 8
```

```
var_dump(null == false); // true
var_dump([] == false); // true
var_dump("1" == "01"); // true (compara como números)
?>
```

Buena práctica: Usa `===` siempre que quieras comparar también el tipo. Evita sorpresas de `==` con 0, "", null y false.

Lógicos

Operador	Nombre	Es true cuando...
<code>&&</code> (o <code>and</code>)	Y	las dos condiciones son true
<code> </code> (o <code>or</code>)	O	al menos una condición es true
<code>!</code>	NO	invierte la condición
<code>xor</code>	O exclusivo	solo una de las dos es true

```
<?php
$edad = 20; $carnet = true;
if ($edad >= 18 && $carnet) { echo "Puede conducir"; }
if ($edad < 12 || $edad > 65) { echo "Tarifa reducida"; }
if (!$carnet) { echo "No tiene carnet"; }
?>
```

Concatenación, ternario y fusión de null

```
<?php
$nombre = "Ana";
echo "Hola " . $nombre; // concatenación con el punto

$mensaje = ($edad >= 18) ? "Mayor" : "Menor"; // operador ternario
$usuario = $_GET["user"] ?? "Invitado"; // ?? devuelve el 2º valor si no existe o es null
?>
```

Precedencia: primero `**`, luego `!`, después `*` / `%`, `+` `-`, comparaciones, `&&`, `||`, `??`, ternario y asignaciones. Ante la duda, usa paréntesis.

4. Salida de datos

echo y print

`echo` envía texto al navegador y admite varios argumentos separados por comas. `print` es muy similar pero admite uno solo y devuelve 1.

```
<?php
$nombre = "Ana";
$edad = 21;
echo "Hola " . $nombre . ", tienes " . $edad . " años."; // concatenar con punto
echo "Hola $nombre, bienvenida."; // interpolar con comillas dobles
echo "Hola ", $nombre, "<br>"; // varios argumentos
print "Esto es print";
?>
```

Comillas simples y dobles

	Comillas dobles " "	Comillas simples ' '
Variables	Se sustituyen: "Ho la \$n" -> Hola Ana	No: 'Ho la \$n' -> Hola \$n
Escapes (\n, \t)	Sí	No (solo \' y \\)
Velocidad	Prácticamente igual	Prácticamente igual

```
<?php
$alumno = ["nombre" => "Ana", "nota" => 8];
echo "Nombre: {$alumno['nombre']}";      // llaves para arrays dentro de comillas dobles
echo "Precio: 5\$";                      // \$ escapa el dólar
echo "Dijo \"hola\"";                    // \" escapa las comillas
echo "Línea 1<br>Línea 2";                // \n NO genera salto en el navegador: usa <br>
?>
```

printf y sprintf

```
<?php
printf("%s tiene %d años y mide %.2f m", "Ana", 21, 1.68); // Ana tiene 21 años y mide 1.68 m
$texto = sprintf("%05d", 42); // "00042" (sprintf devuelve el texto, no lo imprime)
echo $texto;
?>
```

Marcador	Significado
%s	string
%d	entero
%f / %.2f	decimal / con 2 decimales
%05d	entero rellenado con ceros hasta 5 cifras

Depuración: print_r y var_dump

```
<?php
$frutas = ["manzana", "pera"];
echo "<pre>";
print_r($frutas); // Array ( [0] => manzana [1] => pera )
var_dump($frutas); // muestra también tipos y longitudes
echo "</pre>";
?>
```

Heredoc

```
<?php
$nombre = "Ana";
echo <<<EOT
Hola $nombre,
esto respeta los saltos de línea
y permite "comillas" sin escapar.
EOT;
?>
```

5. Estructuras de control

if, elseif y else

```
<?php
$nota = 7;

if ($nota >= 9) {
    echo "Sobresaliente";
} elseif ($nota >= 7) {
    echo "Notable";
} elseif ($nota >= 5) {
    echo "Aprobado";
} else {
    echo "Suspenso";
}

?>
```

- Las condiciones se evalúan **de arriba abajo**: se ejecuta el primer bloque cuya condición sea true y se ignora el resto.
- Se pueden combinar condiciones con **&&**, **||** y **!**.
- Un **if** dentro de otro **if** se llama **if anidado**.
- Cuidado: **if (\$a = 5)** **asigna**, no compara. Para comparar usa **==** o **===**.

Ejemplos típicos

```
<?php
// Año bisiesto: divisible por 4 y no por 100, o divisible por 400
$anio = 2024;
if (($anio % 4 == 0 && $anio % 100 != 0) || $anio % 400 == 0) {
    echo "Bisiesto";
}

// Par o impar
$n = 7;
echo ($n % 2 == 0) ? "Par" : "Impar";

// Mayor de tres números
$a = 4; $b = 9; $c = 6;
if ($a >= $b && $a >= $c) { $mayor = $a; }
elseif ($b >= $a && $b >= $c) { $mayor = $b; }
else { $mayor = $c; }

?>
```

switch

Cuando hay muchas comparaciones sobre la misma variable, **switch** es más legible que encadenar **elseif**. Compara con **==**.

```
<?php
$día = 3;
switch ($día) {
    case 1:
        echo "Lunes";
        break;
    case 2:
        echo "Martes";
        break;
}
```

```

        case 6:
        case 7:                // varios case seguidos comparten el mismo código
            echo "Fin de semana";
            break;
        default:
            echo "Otro día";
    }
?>

```

Importante: Si olvidas **break**, PHP sigue ejecutando los case siguientes (efecto cascada). Es el error más típico con switch.

match (PHP 8)

Es como un switch que **devuelve un valor**, compara con **===** y no necesita **break**.

```

<?php
$dia = 6;
$texto = match($dia) {
    1, 2, 3, 4, 5 => "Laborable",
    6, 7         => "Fin de semana",
    default      => "Día no válido",
};
echo $texto;    // Fin de semana
?>

```

6. Bucles

Bucle	Cuándo usarlo	Comprueba la condición
while	No sabemos cuántas repeticiones habrá	Al principio
do...while	Debe ejecutarse al menos una vez	Al final
for	Sabemos cuántas repeticiones necesitamos	Al principio
foreach	Recorrer los elementos de un array	-

```

<?php
// while
$i = 1;
while ($i <= 5) {
    echo "Número: $i <br>";
    $i++;
} // sin esto el bucle sería infinito

// do...while: se ejecuta al menos una vez
$j = 10;
do {
    echo "j vale $j <br>";
    $j++;
} while ($j < 5); // muestra "j vale 10" una vez

// for (inicio; condición; paso)
for ($k = 1; $k <= 5; $k++) {
    echo "Número: $k <br>";
}

// foreach

```

```

$frutas = ["manzana", "pera", "plátano"];
foreach ($frutas as $fruta) {
    echo $fruta . "<br>";
}
?>

```

break y continue

```

<?php
for ($i = 1; $i <= 10; $i++) {
    if ($i == 3) { continue; }      // salta esta vuelta y sigue con la siguiente
    if ($i == 7) { break; }        // termina el bucle
    echo $i . " ";                // 1 2 4 5 6
}
?>

```

Bucles anidados

```

<?php
// Tablas de multiplicar del 1 al 3
for ($t = 1; $t <= 3; $t++) {
    echo "<b>Tabla del $t</b><br>";
    for ($n = 1; $n <= 10; $n++) {
        echo "$t x $n = " . ($t * $n) . "<br>";
    }
}

// Triángulo de asteriscos
for ($fila = 1; $fila <= 5; $fila++) {
    for ($col = 1; $col <= $fila; $col++) {
        echo "*";
    }
    echo "<br>";
}
?>

```

Patrones típicos de examen

```

<?php
// Acumulador: suma de 1 a 100
$suma = 0;
for ($i = 1; $i <= 100; $i++) { $suma += $i; }

// Factorial de 6
$fact = 1; $n = 6;
while ($n > 1) { $fact *= $n; $n--; }

// Contador: cuántos pares hay entre 1 y 50
$pares = 0;
for ($i = 1; $i <= 50; $i++) { if ($i % 2 == 0) { $pares++; } }

// Cuenta atrás
for ($i = 10; $i >= 1; $i--) { echo "$i<br>"; }
echo "¡Despegue!";

// Número primo
$num = 17; $esPrimo = $num > 1;
for ($d = 2; $d * $d <= $num; $d++) {
    if ($num % $d == 0) { $esPrimo = false; break; }
}
?>

```

7. Arrays

Un array almacena **varios valores en una sola variable**. Pueden ser **indexados** (por posición numérica, empezando en 0) o **asociativos** (por clave de texto).

```
<?php
// Array indexado
$frutas = array("manzana", "pera", "plátano"); // o con corchetes: ["manzana", "pera", "plátano"]
echo $frutas[0]; // manzana
$frutas[] = "kiwi"; // añade al final
$frutas[1] = "melón"; // modifica la posición 1
unset($frutas[2]); // elimina (no reindexa)

// Array asociativo
$alumno = array("nombre" => "Ana", "edad" => 21);
echo $alumno["nombre"]; // Ana
$alumno["ciudad"] = "Santander"; // añade clave nueva
$alumno["edad"] = 22; // modifica

echo count($frutas); // número de elementos
print_r($alumno); // vuelca el contenido (depuración)
?>
```

Recorrer arrays

```
<?php
$frutas = ["manzana", "pera", "plátano"];

foreach ($frutas as $fruta) { // solo valores
    echo $fruta . "<br>";
}
for ($i = 0; $i < count($frutas); $i++) { // con índice numérico
    echo $i . ": " . $frutas[$i] . "<br>";
}

$alumno = ["nombre" => "Ana", "edad" => 21];
foreach ($alumno as $clave => $valor) { // claves y valores
    echo "$clave: $valor<br>";
}
?>
```

Arrays multidimensionales

```
<?php
$alumnos = [
    ["nombre" => "Ana", "nota" => 8],
    ["nombre" => "Luis", "nota" => 4],
    ["nombre" => "Eva", "nota" => 6.5],
];
echo $alumnos[0]["nombre"]; // Ana

foreach ($alumnos as $alumno) {
    echo $alumno["nombre"] . ": " . $alumno["nota"] . "<br>";
}

// Media y aprobados
$suma = 0;
foreach ($alumnos as $a) {
    $suma += $a["nota"];
    if ($a["nota"] >= 5) { echo $a["nombre"] . " aprobó<br>"; }
}
```

```

    echo "Media: " . ($suma / count($alumnos));
?>

```

Funciones de arrays

Función	Qué hace	Ejemplo
<code>count(\$a)</code>	Nº de elementos	<code>count([4,5,6]) -> 3</code>
<code>array_push(\$a, x)</code>	Añade al final	equivale a <code>\$a[] = x</code>
<code>array_pop(\$a)</code>	Quita y devuelve el último	<code>array_pop(\$a)</code>
<code>array_shift(\$a)</code>	Quita y devuelve el primero	<code>array_shift(\$a)</code>
<code>array_unshift(\$a, x)</code>	Añade al principio	<code>array_unshift(\$a, 0)</code>
<code>in_array(x, \$a)</code>	¿Existe el valor?	<code>in_array("pera", \$f) -> true</code>
<code>array_search(x, \$a)</code>	Clave del valor (o false)	<code>array_search("pera", \$f)</code>
<code>array_key_exists(k, \$a)</code>	¿Existe la clave?	<code>array_key_exists("nombre", \$a1)</code>
<code>array_keys(\$a)</code>	Array con las claves	<code>array_keys(\$alumno)</code>
<code>array_values(\$a)</code>	Array con los valores	<code>array_values(\$alumno)</code>
<code>array_merge(\$a, \$b)</code>	Une arrays	<code>array_merge([1,2],[3]) -> 1,2,3</code>
<code>array_slice(\$a, ini, n)</code>	Extrae un trozo	<code>array_slice(\$a, 1, 2)</code>
<code>array_reverse(\$a)</code>	Invierte el orden	<code>array_reverse([1,2,3]) -> 3,2,1</code>
<code>array_unique(\$a)</code>	Elimina repetidos	<code>array_unique([1,1,2]) -> 1,2</code>
<code>array_sum(\$a)</code>	Suma de valores	<code>array_sum([1,2,3]) -> 6</code>
<code>array_product(\$a)</code>	Producto de valores	<code>array_product([2,3]) -> 6</code>
<code>max(\$a) / min(\$a)</code>	Mayor / menor	<code>max([3,9,4]) -> 9</code>
<code>range(a, b)</code>	Array de a hasta b	<code>range(1,5) -> 1,2,3,4,5</code>
<code>implode(sep, \$a)</code>	Array a texto	<code>implode("-", [1,2]) -> "1-2"</code>
<code>explode(sep, txt)</code>	Texto a array	<code>explode(",", "a,b") -> [a,b]</code>
<code>array_map(fn, \$a)</code>	Aplica función a cada elemento	<code>array_map(fn(\$x)=>\$x*2, \$a)</code>
<code>array_filter(\$a, fn)</code>	Filtra elementos	<code>array_filter(\$a, fn(\$x)=>\$x>5)</code>

Ordenar arrays

Función	Ordena por	Orden	Mantiene claves
<code>sort(\$a)</code>	valor	ascendente	No (reindexa)
<code>rsort(\$a)</code>	valor	descendente	No (reindexa)
<code>asort(\$a)</code>	valor	ascendente	Sí
<code>arsort(\$a)</code>	valor	descendente	Sí
<code>ksort(\$a)</code>	clave	ascendente	Sí
<code>krsort(\$a)</code>	clave	descendente	Sí

Importante: Las funciones de ordenación modifican el array original (se pasa por referencia). No hay que hacer `$a = sort($a)`.

Ejemplos típicos

```
<?php
$numeros = [4, 9, 2, 15, 7, 12];

// Suma, media, mayor y menor sin funciones de PHP
$suma = 0; $mayor = $numeros[0]; $menor = $numeros[0];
foreach ($numeros as $n) {
    $suma += $n;
    if ($n > $mayor) { $mayor = $n; }
    if ($n < $menor) { $menor = $n; }
}
$media = $suma / count($numeros);

// Buscar un valor con una bandera
$encontrado = false;
foreach (["manzana", "pera"] as $f) {
    if ($f === "pera") { $encontrado = true; }
}

// Nuevo array solo con los pares
$pares = [];
foreach ($numeros as $n) {
    if ($n % 2 == 0) { $pares[] = $n; }
}

// Recorrer al revés
for ($i = count($numeros) - 1; $i >= 0; $i--) { echo $numeros[$i] . " "; }
?>
```

8. Funciones

Una función agrupa un **bloque de código reutilizable**. Puede recibir parámetros y devolver un valor con `return`.

```
<?php
function saludar($nombre, $formal = false) {
    if ($formal) {
        return "Estimado/a $nombre";
    }
    return "¡Hola $nombre!";
}

echo saludar("Ana");           // ¡Hola Ana!
echo saludar("Sr. Pérez", true); // Estimado/a Sr. Pérez
?>
```

- Se definen con `function nombre(parámetros) { ... }` y se llaman con `nombre(argumentos)`.
- Un parámetro con **valor por defecto** (`$formal = false`) es opcional al llamar a la función.
- `return` devuelve un valor y **termina** la función. Sin return, devuelve null.
- Para devolver varios valores, devuelve un array.

Tipos en parámetros y retorno

```
<?php
function sumar(int $a, int $b): int {
    return $a + $b;
}
function esPar(int $n): bool {
    return $n % 2 == 0;
}
function calcularIVA(float $precio, int $iva = 21): float {
    return $precio * (1 + $iva / 100);
}
function sumaArray(array $numeros): int|float {
    $suma = 0;
    foreach ($numeros as $n) { $suma += $n; }
    return $suma;
}

echo sumar(3, 4);           // 7
var_dump(esPar(10));       // bool(true)
echo calcularIVA(100);     // 121
echo sumaArray([1, 2, 3]); // 6
?>
```

Ámbito de las variables

```
<?php
$x = 10; // variable global

function prueba() {
    // echo $x; // ERROR: dentro de la función no existe $x
    global $x; // con global se puede usar (mejor pasar parámetros)
    $x++;
}

function contador() {
    static $veces = 0; // conserva su valor entre llamadas
    $veces++;
    return $veces;
}
echo contador(); // 1
echo contador(); // 2
?>
```

Paso por valor y por referencia

```
<?php
function sumaUno($n) { $n++; } // por valor: no cambia el original
function sumaUnoRef(&$n) { $n++; } // por referencia (&): cambia el original

$a = 5;
sumaUno($a); echo $a; // 5
sumaUnoRef($a); echo $a; // 6
?>
```

Recursividad, funciones anónimas y argumentos variables

```
<?php
function factorial($n) {
    if ($n <= 1) { return 1; }
    return $n * factorial($n - 1);    // la función se llama a sí misma
}
echo factorial(5);                  // 120

$doble = fn($x) => $x * 2;           // función flecha
echo $doble(4);                    // 8

$cuadrado = function($x) { return $x * $x; }; // función anónima
echo $cuadrado(3);                 // 9

function sumaTodo(...$nums) {      // número variable de argumentos
    return array_sum($nums);
}
echo sumaTodo(1, 2, 3, 4);          // 10
?>
```

Incluir otros archivos

Instrucción	Si el archivo no existe
<code>include "cabecera.php";</code>	Aviso (warning) y el script continúa
<code>require "cabecera.php";</code>	Error fatal y el script se detiene
<code>include_once / require_once</code>	Igual, pero no lo carga dos veces

Funciones matemáticas útiles

Función	Resultado	Función	Resultado
<code>abs(-5)</code>	5	<code>sqrt(16)</code>	4
<code>round(3.14159, 2)</code>	3.14	<code>pow(2, 3)</code>	8
<code>floor(3.7)</code>	3	<code>intdiv(7, 2)</code>	3
<code>ceil(3.2)</code>	4	<code>fmod(7, 2)</code>	1
<code>max(3, 9, 4)</code>	9	<code>rand(1, 10)</code>	entero aleatorio
<code>min(3, 9, 4)</code>	3	<code>pi()</code> o <code>M_PI</code>	3.14159...
<code>number_format(1234.567, 2, ",", ".")</code>	1.234,57	<code>is_numeric("12.5")</code>	true

Fecha y hora

```
<?php
date_default_timezone_set("Europe/Madrid");
echo date("d/m/Y");           // 07/10/2026
echo date("H:i:s");           // 18:30:05
echo date("Y-m-d H:i");       // 2026-10-07 18:30
echo time();                  // segundos desde 1970 (marca de tiempo)
$hora = (int) date("H");       // hora actual como número
?>
```

Letra	Significado	Letra	Significado
d	Día 01-31	H	Hora 00-23
j	Día sin cero inicial	i	Minutos 00-59
m	Mes 01-12	s	Segundos 00-59
n	Mes sin cero inicial	D / L	Día de la semana corto / largo (en inglés)
Y	Año con 4 cifras	N	Día de la semana 1 (lunes) a 7 (domingo)

9. Mezclando PHP y HTML

La verdadera utilidad de PHP aparece al combinarlo con HTML para generar **contenido dinámico**. Se puede entrar y salir del modo PHP tantas veces como haga falta dentro de la misma página.

```
<!DOCTYPE html>
<html>
<body>
  <h1>Lista de la compra</h1>
  <ul>
    <?php
      $productos = array("Leche", "Pan", "Huevos");
      foreach ($productos as $producto) {
        echo "<li>$producto</li>";
      }
    ?>
  </ul>
</body>
</html>
```

Las etiquetas `<li>` se generan dinámicamente, una por cada elemento del array: si añadimos un producto más, aparece en la lista sin tocar el HTML.

Tabla generada con un array

```
<?php
  $alumnos = [ ["nombre" => "Ana", "nota" => 8], ["nombre" => "Luis", "nota" => 4] ];
?>
<table border="1">
  <tr><th>Nombre</th><th>Nota</th></tr>
  <?php foreach ($alumnos as $a) {
    $color = ($a["nota"] >= 5) ? "lightgreen" : "salmon";
    echo "<tr style='background:$color'>";
    echo "<td>" . $a["nombre"] . "</td><td>" . $a["nota"] . "</td>";
    echo "</tr>";
  } ?>
</table>
```

Select y lista numerada

```
<?php $meses = ["Enero", "Febrero", "Marzo", "Abril"]; ?>
<select name="mes">
  <?php foreach ($meses as $i => $mes) { ?>
    <option value="<?php echo $i + 1; ?>"><?php echo $mes; ?></option>
  <?php } ?>
</select>

<ol>
```

```
<?php for ($i = 1; $i <= 5; $i++) { echo "<li>Elemento $i</li>"; } ?>
</ol>
```

Sintaxis alternativa

Cuando se mezcla mucho HTML, es más legible usar `:` y `endif`, `endforeach`, `endfor`, `endwhile` en lugar de llaves, junto con `<?= ?>` para imprimir.

```
<?php $hora = 15; ?>
<?php if ($hora < 12): ?>
    <h2>Buenos días</h2>
<?php elseif ($hora < 20): ?>
    <h2>Buenas tardes</h2>
<?php else: ?>
    <h2>Buenas noches</h2>
<?php endif; ?>

<ul>
<?php foreach ($productos as $p): ?>
    <li><?= $p ?></li>
<?php endforeach; ?>
</ul>
```

10. Formularios: \$_GET y \$_POST

PHP recoge automáticamente los datos enviados desde un formulario HTML en variables especiales llamadas **superglobales**. Cada campo se identifica por su atributo `name`.

Superglobal	Contiene
<code>\$_GET</code>	Datos enviados con <code>method="get"</code> (van en la URL)
<code>\$_POST</code>	Datos enviados con <code>method="post"</code> (van en el cuerpo de la petición)
<code>\$_REQUEST</code>	GET, POST y cookies juntos (mejor evitarlo)
<code>\$_SERVER</code>	Información del servidor y de la petición, p. ej. <code>\$_SERVER["REQUEST_METHOD"]</code>
<code>\$_FILES</code>	Archivos subidos
<code>\$_SESSION</code> , <code>\$_COOKIE</code>	Sesión y cookies

GET frente a POST

	GET	POST
Datos	Visibles en la URL (<code>?nombre=Ana</code>)	No visibles en la URL
Longitud	Limitada	Prácticamente sin límite
Marcadores/compartir	Se puede guardar o compartir el enlace	No
Uso adecuado	Búsquedas, filtros, consultas	Login, registros, datos sensibles, formularios que modifican datos

Primer formulario con isset()

```
<form method="get" action="">
    Nombre: <input type="text" name="nombre">
    <input type="submit" value="Enviar">
</form>

<?php
    if (isset($_GET["nombre"])) {
        echo "Hola " . $_GET["nombre"];
    }
?>
```

`isset()` comprueba si una variable existe (en este caso, si el formulario ya se ha enviado) antes de intentar usarla, evitando errores como `*Undefined array key*`.

Detectar que se ha enviado el formulario

```
<?php
    if ($_SERVER["REQUEST_METHOD"] === "POST") {
        // el formulario se ha enviado: procesar los datos aquí
    }
    // otra opción: if (isset($_POST["enviar"])) con <input type="submit" name="enviar">
?>
```

Todos los tipos de campo

```
<form method="post" action="">
    Nombre: <input type="text" name="nombre"><br>
    Edad: <input type="number" name="edad"><br>
    Sexo: <input type="radio" name="sexo" value="H"> Hombre
          <input type="radio" name="sexo" value="M"> Mujer<br>
    Aficiones: <input type="checkbox" name="aficiones[]" value="cine"> Cine
               <input type="checkbox" name="aficiones[]" value="musica"> Música<br>
    País: <select name="pais">
           <option value="es">España</option>
           <option value="pt">Portugal</option>
        </select><br>
    Comentarios: <textarea name="comentarios"></textarea><br>
    <input type="hidden" name="origen" value="web">
    <input type="submit" value="Enviar">
</form>

<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nombre = $_POST["nombre"] ?? "";
    $sexo = $_POST["sexo"] ?? "no indicado"; // un radio sin marcar no se envía
    $aficiones = $_POST["aficiones"] ?? []; // los checkbox llegan como array
    $pais = $_POST["pais"];

    echo "Aficiones: " . implode(", ", $aficiones);
}
?>
```

- **Radio y checkbox** sin marcar **no se envían**: hay que usar `isset()` o `??`.
- Los checkbox con el mismo nombre terminado en `[]` llegan como un **array**.
- Todo lo que llega de un formulario es **texto** (string), incluso en `type="number"`. Convierte con `(int)` o `(float)` si hace falta.
- `action=""` envía el formulario a la misma página.

Ejemplo: calculadora

```
<form method="post" action="">
  <input type="number" name="a"> <input type="number" name="b">
  <select name="op">
    <option value="suma">+</option>
    <option value="resta">-</option>
    <option value="mult">*</option>
    <option value="div">/</option>
  </select>
  <input type="submit" value="Calcular">
</form>

<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
  $a = (float) $_POST["a"];
  $b = (float) $_POST["b"];
  $op = $_POST["op"];
  switch ($op) {
    case "suma": echo $a + $b; break;
    case "resta": echo $a - $b; break;
    case "mult": echo $a * $b; break;
    case "div":
      echo ($b == 0) ? "No se puede dividir entre 0" : $a / $b;
      break;
  }
}
?>
```

Recordar lo escrito (formulario persistente)

```
<?php $nombre = $_POST["nombre"] ?? ""; ?>
<form method="post" action="">
  Nombre: <input type="text" name="nombre" value="<?php echo htmlspecialchars($nombre); ?>">
  <input type="submit" value="Enviar">
</form>
```

11. Validación y sanitización de datos

Nunca hay que fiarse de lo que envía el usuario. Hay dos tareas distintas:

Tarea	Qué hace	Ejemplo
Validar	Comprueba que el dato tiene el formato correcto y lo acepta o lo rechaza	¿Es un email válido? ¿La edad está entre 0 y 120?
Sanear (sanitizar)	Limpia o transforma el dato para que sea seguro	Quitar espacios, convertir < en <

Comprobar que el dato existe y no está vacío

```
<?php
isset($_POST["nombre"]); // ¿existe el campo?
empty($_POST["nombre"]); // true si no existe o es "", "0", 0, null, false, []
$nombre = trim($_POST["nombre"] ?? ""); // valor por defecto y sin espacios laterales

if ($nombre === "") { echo "El nombre es obligatorio"; }
if (strlen($nombre) < 3) { echo "Mínimo 3 caracteres"; }
if (is_numeric($_POST["edad"])) { echo "Es un número"; }

?>
```

Importante: `empty("0")` es true: un campo con el valor 0 se consideraría vacío. Para campos numéricos compara con `=== ""` en lugar de usar `empty()`.

filter_var(): validar

`filter_var($valor, FILTRO, opciones)` devuelve el valor si es válido o **false** si no lo es. Compara siempre con `=== false` (un 0 válido también es "falso").

Filtro de validación	Comprueba
<code>FILTER_VALIDATE_INT</code>	Entero (admite <code>min_range</code> y <code>max_range</code>)
<code>FILTER_VALIDATE_FLOAT</code>	Número decimal
<code>FILTER_VALIDATE_EMAIL</code>	Email con formato válido
<code>FILTER_VALIDATE_URL</code>	URL válida
<code>FILTER_VALIDATE_IP</code>	Dirección IP
<code>FILTER_VALIDATE_BOOLEAN</code>	true/false, on/off, yes/no, 1/0
<code>FILTER_VALIDATE_REGEXP</code>	Cumple una expresión regular

```
<?php
$email = "ana@correo.com";
if (filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
    echo "Email no válido";
}

$edad = "25";
$opciones = ["options" => ["min_range" => 0, "max_range" => 120]];
if (filter_var($edad, FILTER_VALIDATE_INT, $opciones) === false) {
    echo "La edad debe ser un entero entre 0 y 120";
}

var_dump(filter_var("abc", FILTER_VALIDATE_INT));           // bool(false)
var_dump(filter_var("12.5", FILTER_VALIDATE_FLOAT));        // float(12.5)
var_dump(filter_var("http://x.es", FILTER_VALIDATE_URL));   // string
?>
```

Sanear: filter_var() y funciones de limpieza

Función / filtro	Qué hace
<code>trim(\$s)</code>	Elimina espacios (y saltos de línea) al principio y al final
<code>stripslashes(\$s)</code>	Quita las barras invertidas
<code>strip_tags(\$s)</code>	Elimina las etiquetas HTML y PHP
<code>htmlspecialchars(\$s)</code>	Convierte <code>&</code> <code><</code> <code>></code> <code>"</code> <code>'</code> en entidades HTML. Evita ataques XSS
<code>FILTER_SANITIZE_EMAIL</code>	Elimina caracteres no permitidos en un email
<code>FILTER_SANITIZE_URL</code>	Elimina caracteres no permitidos en una URL
<code>FILTER_SANITIZE_NUMBER_INT</code>	Deja solo dígitos y los signos + -
<code>FILTER_SANITIZE_SPECIAL_CHARS</code>	Escapa caracteres especiales HTML
<code>FILTER_SANITIZE_FULL_SPECIAL_CHARS</code>	Equivalente a <code>htmlspecialchars</code> con comillas

Importante: `FILTER_SANITIZE_STRING` está obsoleto desde PHP 8.1. Usa `htmlspecialchars()` para mostrar datos y `strip_tags()` si quieres eliminar etiquetas.

```
<?php
$texto = " <script>alert('hack')</script> Hola ";
echo trim($texto);           // quita los espacios laterales
echo htmlspecialchars($texto); // se muestra como texto, no se ejecuta
echo strip_tags($texto);      // alert('hack') Hola

echo filter_var("ana(@)correo.com", FILTER_SANITIZE_EMAIL); // ana@correo.com
echo filter_var("a1b2c3", FILTER_SANITIZE_NUMBER_INT);      // 123
?>
```

Función de limpieza reutilizable

```
<?php
function limpiar($dato) {
    $dato = trim($dato);
    $dato = stripslashes($dato);
    $dato = htmlspecialchars($dato);
    return $dato;
}
$nombre = limpiar($_POST["nombre"] ?? "");
?>
```

Expresiones regulares básicas

```
<?php
// Solo letras y espacios (con acentos y ñ). /u = UTF-8
if (!preg_match("/^[a-zA-ZáéíóúÁÉÍÓÚÑñ ]+$/u", $nombre)) {
    echo "Solo se permiten letras y espacios";
}
// Teléfono de 9 cifras
if (!preg_match("/^[0-9]{9}$/", $telefono)) { echo "Teléfono no válido"; }
?>
```

Patrón	Significado
<code>^</code> y <code>\$</code>	Inicio y fin del texto
<code>[a-z]</code> , <code>[0-9]</code>	Un carácter del rango
<code>+</code> , <code>*</code> , <code>?</code>	Una o más veces, cero o más, cero o una
<code>{9}</code> , <code>{3,8}</code>	Exactamente 9 veces, entre 3 y 8

Formulario completo con validación

```
<?php
function limpiar($dato) {
    return htmlspecialchars(stripslashes(trim($dato)));
}

$errores = [];
$nombre = $email = $edad = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nombre = limpiar($_POST["nombre"] ?? "");
    $email = limpiar($_POST["email"] ?? "");
    $edad = limpiar($_POST["edad"] ?? "");
}
```

```

if ($nombre === "") {
    $errores[] = "El nombre es obligatorio.";
} elseif (strlen($nombre) < 3) {
    $errores[] = "El nombre debe tener al menos 3 caracteres.";
}
if (filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
    $errores[] = "El email no es válido.";
}
$range = ["options" => ["min_range" => 0, "max_range" => 120]];
if (filter_var($edad, FILTER_VALIDATE_INT, $range) === false) {
    $errores[] = "La edad debe ser un entero entre 0 y 120.";
}
}
?>
<form method="post" action="">
    Nombre: <input type="text" name="nombre" value="<?php echo $nombre; ?>"><br>
    Email: <input type="text" name="email" value="<?php echo $email; ?>"><br>
    Edad: <input type="text" name="edad" value="<?php echo $edad; ?>"><br>
    <input type="submit" value="Enviar">
</form>
<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    if (count($errores) > 0) {
        echo "<ul>";
        foreach ($errores as $error) { echo "<li>$error</li>"; }
        echo "</ul>";
    } else {
        echo "Datos correctos: $nombre, $email, $edad";
    }
}
?>

```

Contraseña y confirmación

```

<?php
$clave = $_POST["clave"] ?? "";
$clave2 = $_POST["clave2"] ?? "";
if (strlen($clave) < 8) { $errores[] = "Mínimo 8 caracteres."; }
if ($clave !== $clave2) { $errores[] = "Las contraseñas no coinciden."; }
?>

```

Idea clave: Validar en el navegador (atributos HTML `required`, `type="email"`, `min`, `max`) mejora la experiencia, pero **siempre hay que validar también en el servidor**: el usuario puede saltarse el HTML.

12. Cadenas de texto

Función	Qué hace	Ejemplo -> resultado
<code>strlen(\$s)</code>	Longitud en bytes	<code>strlen("Hola mundo")</code> -> 10
<code>mb_strlen(\$s)</code>	Longitud en caracteres (UTF-8)	<code>mb_strlen("plátano")</code> -> 7
<code>strtoupper(\$s)</code>	A mayúsculas	<code>strtoupper("hola")</code> -> HOLA
<code>strtolower(\$s)</code>	A minúsculas	<code>strtolower("HOLA")</code> -> hola
<code>ucfirst(\$s)</code>	Primera letra en mayúscula	<code>ucfirst("hola mundo")</code> -> Hola mundo
<code>ucwords(\$s)</code>	Cada palabra con mayúscula	<code>ucwords("hola mundo")</code> -> Hola Mundo

Función	Qué hace	Ejemplo -> resultado
<code>lcfirst(\$s)</code>	Primera letra en minúscula	<code>lcfirst("Hola")</code> -> hola
<code>trim(\$s)</code>	Quita espacios laterales	<code>trim(" hi ")</code> -> hi
<code>ltrim</code> / <code>rtrim</code>	Solo a la izquierda / derecha	<code>rtrim("hi ")</code> -> hi
<code>str_replace(a, b, \$s)</code>	Sustituye texto	<code>str_replace("PHP", "JS", "Uso PHP")</code> -> Uso JS
<code>substr(\$s, ini, n)</code>	Extrae un trozo	<code>substr("Hola mundo", 0, 4)</code> -> Hola
<code>substr(\$s, -n)</code>	Los últimos n caracteres	<code>substr("Hola mundo", -5)</code> -> mundo
<code>strpos(\$s, b)</code>	Posición de b (o false)	<code>strpos("Hola mundo", "mundo")</code> -> 5
<code>stripos(\$s, b)</code>	Igual, sin distinguir mayúsculas	<code>stripos("Hola", "HOLA")</code> -> 0
<code>str_contains(\$s, b)</code>	¿Contiene b? (PHP 8)	<code>str_contains("Hola", "ol")</code> -> true
<code>str_starts_with(\$s, b)</code>	¿Empieza por b? (PHP 8)	<code>str_starts_with("Hola", "Ho")</code> -> true
<code>str_ends_with(\$s, b)</code>	¿Acaba en b? (PHP 8)	<code>str_ends_with("Hola", "la")</code> -> true
<code>strrev(\$s)</code>	Invierte la cadena	<code>strrev("abc")</code> -> cba
<code>str_repeat(\$s, n)</code>	Repite la cadena	<code>str_repeat("ab", 3)</code> -> ababab
<code>str_pad(\$s, n, c, tipo)</code>	Rellena hasta n caracteres	<code>str_pad("7", 3, "0", STR_PAD_LEFT)</code> -> 007
<code>strcmp(\$a, \$b)</code>	Compara (0 si son iguales)	<code>strcmp("a", "a")</code> -> 0
<code>strcasecmp(\$a, \$b)</code>	Compara sin distinguir mayúsculas	<code>strcasecmp("A", "a")</code> -> 0
<code>explode(sep, \$s)</code>	Texto a array	<code>explode(",", "a,b,c")</code> -> [a, b, c]
<code>implode(sep, \$a)</code>	Array a texto	<code>implode(" - ", ["a", "b"])</code> -> a - b
<code>str_word_count(\$s)</code>	Cuenta palabras	<code>str_word_count("Hola buen mundo")</code> -> 3
<code>substr_count(\$s, b)</code>	Veces que aparece b	<code>substr_count("banana", "a")</code> -> 3
<code>nl2br(\$s)</code>	Saltos de línea a <code>
</code>	<code>nl2br("a\nb")</code>
<code>wordwrap(\$s, n)</code>	Parte el texto cada n caracteres	<code>wordwrap("Hola mundo", 5, "
")</code>

Importante: `strpos()` devuelve **0** si la palabra está al principio, y 0 es "falso". Compara siempre con `!== false`: `if (strpos($s, "a") !== false)`. En PHP 8 es más cómodo `str_contains()`.

Acceder a los caracteres y recorrer una cadena

```
<?php
$palabra = "Hola";
echo $palabra[0];           // H (primer carácter)
echo $palabra[strlen($palabra) - 1]; // a (último carácter)

for ($i = 0; $i < strlen($palabra); $i++) {
    echo $palabra[$i] . "-"; // H-o-l-a-
}
?>
```

Ejemplos típicos

```
<?php
// Contar vocales
$frase = "Me gusta programar en PHP";
$vocales = 0;
for ($i = 0; $i < strlen($frase); $i++) {
    if (str_contains("aeiouAEIOU", $frase[$i])) { $vocales++; }
}

// Palíndromo
$palabra = "reconocer";
echo ($palabra === strrev($palabra)) ? "Es palíndromo" : "No lo es";

// Primera letra de cada palabra en mayúscula y contar palabras
echo ucwords(strtolower("hOLA mUNDO")); // Hola Mundo
echo str_word_count($frase); // 5

// Dividir y unir
$lista = explode(",", "manzana,pera,plátano");
foreach ($lista as $fruta) { echo $fruta . "<br>"; }
echo implode(" - ", $lista); // manzana - pera - plátano

// Sustituir y buscar
echo str_replace("PHP", "JavaScript", $frase);
echo strpos($frase, "PHP"); // 22
?>
```

Importante: `strlen()` cuenta **bytes**. Con UTF-8 una letra con acento ocupa 2 bytes, así que `strlen("plátano")` da 8 y `mb_strlen("plátano")` da 7. Usa las versiones `mb_` (`mb_strtoupper`, `mb_substr`...) con texto con acentos.

13. Resumen, errores típicos y buenas prácticas

Buenas prácticas

- Usa **nombres de variables descriptivos** (`$precioTotal` en lugar de `$x`).
- **Comenta** el código cuando la lógica no sea evidente a simple vista.
- Utiliza `**===**` en lugar de `==` cuando quieras comparar también el tipo de dato.
- **Indenta** el código dentro de llaves `{ }` para que sea legible.
- Comprueba siempre con `**isset()*` (o `??`) si una variable existe antes de usarla, especialmente con `$_GET` y `$_POST`.
- **Valida y sanea** todos los datos del usuario. Muestra siempre con `htmlspecialchars()`.
- Prueba el código poco a poco: escribe unas líneas, ejecuta y comprueba.

Errores típicos

Error	Causa	Solución
Parse error: syntax error	Falta un <code>;</code> , una llave <code>}</code> o unas comillas	Revisar la línea indicada y la anterior
Undefined variable <code>\$x</code>	Variable sin declarar o mal escrita (mayúsculas)	Declararla antes y revisar el nombre

Error	Causa	Solución
Undefined array key	Clave o índice que no existe (típico con <code>\$_POST</code>)	<code>isset()</code> o <code>??</code>
Texto con <code>\$nombre</code> literal	Se usaron comillas simples	Usar comillas dobles
El <code>if</code> siempre se cumple	Se escribió <code>=</code> en vez de <code>==</code> o <code>===</code>	Usar <code>===</code> para comparar
Bucle infinito	No se modifica la variable de la condición	Añadir <code>\$i++</code> o revisar la condición
El switch ejecuta varios casos	Falta <code>break</code>	Añadir <code>break;</code> en cada case
<code>strpos</code> falla con 0	Se usó <code>if (strpos(...))</code>	Comparar con <code>!== false</code>
Los espacios del <code>\$_POST</code>	El usuario escribió espacios	<code>trim()</code>
No se ve el salto de línea	<code>\n</code> no es un salto en HTML	Usar <code>
</code>
Division by zero	Divisor igual a 0	Comprobar <code>\$b != 0</code> antes

Chuleta de estructuras

```
<?php
    if (cond) { } elseif (cond) { } else { }
    switch ($v) { case 1: ...; break; default: ...; }
    while (cond) { }      do { } while (cond);
    for ($i = 0; $i < $n; $i++) { }
    foreach ($array as $clave => $valor) { }
    function nombre($a, $b = 0) { return $a + $b; }

    $_GET["campo"]    $_POST["campo"]    $_POST["campo"] ?? "defecto"
    isset()  empty()  trim()  htmlspecialchars()  filter_var()
?>
```